Team 18: Video and Run Guide

Concept Drift Detection in Social Media Streams · IOE413 Streaming Data Analytics · IIIT Kottayam
Bichu Devnarayan (2023BCS0102) · Albin John (2023BCD0005) · Sarthak (2023BCS0068) · Aswin Paramesh (2023BCD0024)

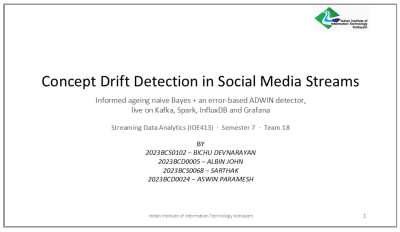
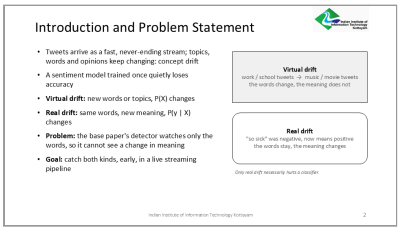
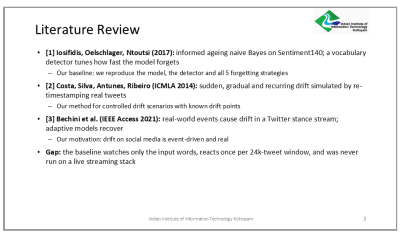
What the video must be (LMS instructions)

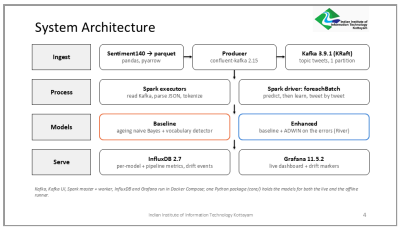
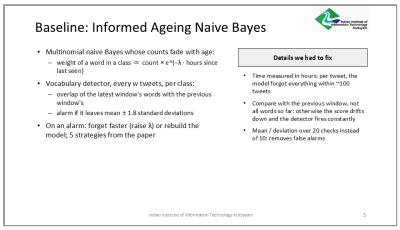
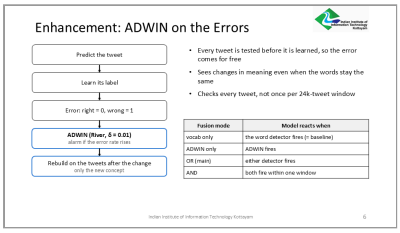
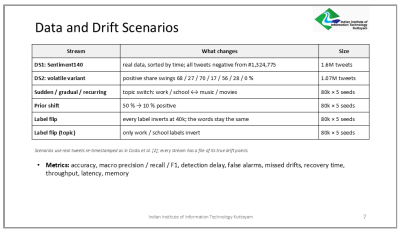
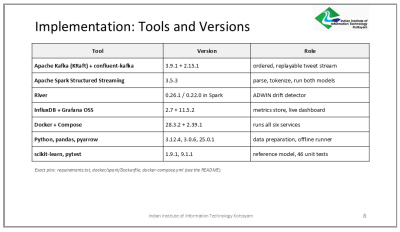
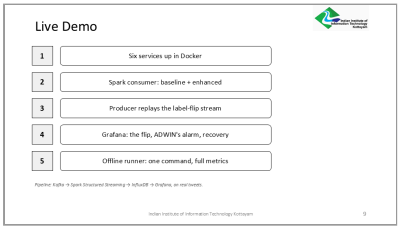
- **MP4** only, **at most 6:00** long, with **audio narration**.
- Must contain the **PPT presentation** and a **demonstration of the complete project workflow** (the pipeline actually running end to end on streaming data).
- Every submitted file starts with the team number: `18_` (for example `18_Demo_Video.mp4`).
- Only the **team lead** uploads to the LMS. Course timeline: final submission, presentation and Q&A in the week of **15–21 October 2026**.

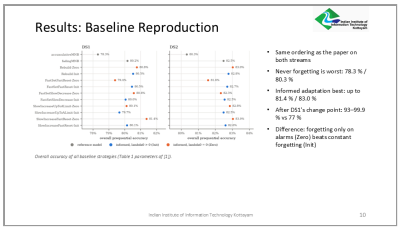
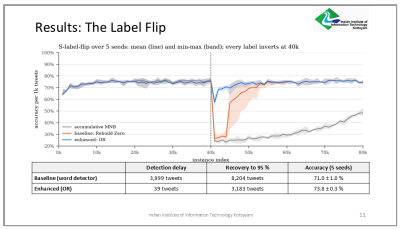
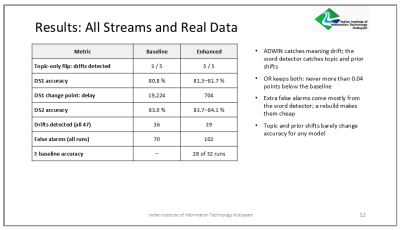
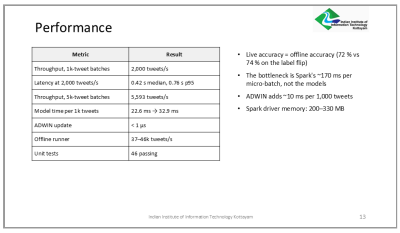
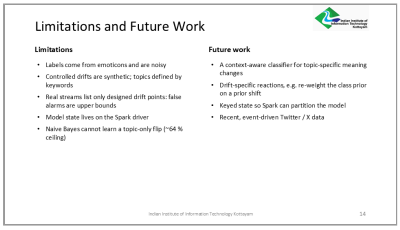
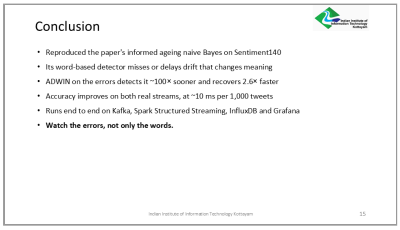
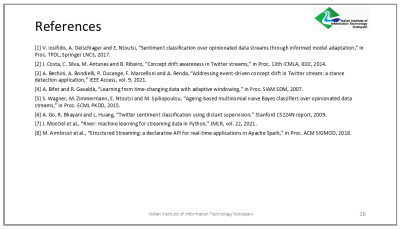
This guide gives the **route** of the video (Section 1), the **demo steps with exact commands** (Section 2), **how to record and export** it (Section 3), **how to set up and run the project from scratch** on any machine (Section 4), troubleshooting (Section 5) and the final checklist (Section 6). Planned length: slides 4:02 + demo 1:35 = **5:37**, leaving 23 s of margin.

1. Route of the video (what to show and what to say)

Read the narration at a calm pace (about 140 words per minute). The same text is in the speaker notes of `Presentation.pptx` (View → Notes, or Presenter View during the slide show). The speaker split is a suggestion that gives each member a part; one narrator is also fine.

Start	Screen	Slide, speaker and narration
0:00 12 s		**Slide 1: Title** Speaker: Bichu Devnarayan *"Hello, we are Team 18: Bichu Devnarayan, Albin John, Sarthak and Aswin Paramesh. Our project is concept drift detection in social media streams."*
0:12 22 s		**Slide 2: Introduction and Problem Statement** Speaker: Bichu Devnarayan *"Tweets arrive as a fast, never-ending stream, and topics, words and opinions keep changing: concept drift. Some drift only changes the words, like a switch from work tweets to music tweets. Real drift changes the meaning: the same words now signal a different sentiment, and a model trained once quietly loses accuracy."*
0:34 18 s		**Slide 3: Literature Review** Speaker: Bichu Devnarayan *"We built on three papers. Iosifidis and colleagues adapt a naive Bayes classifier when a word-based detector signals change; that is our baseline. Costa and colleagues show how to simulate drift with real tweets, and Bechini and colleagues show that real events cause it. The gap: the baseline only watches the words."*

0:52 **22 s**		**Slide 4: System Architecture** Speaker: Albin John *"Here is the pipeline. A Python producer replays Sentiment140's 1.6 million tweets, in time order, into Kafka. Spark Structured Streaming tokenizes each micro-batch, and the driver runs the baseline and our enhanced model side by side. Metrics and drift events go to InfluxDB and Grafana, all in Docker."*
1:14 **20 s**		**Slide 5: Baseline: Informed Ageing Naive Bayes** Speaker: Albin John *"The baseline is an ageing naive Bayes: old word counts fade with time. Every few thousand tweets a detector compares the words of recent tweets with the previous window, and when they change, the model forgets faster or is rebuilt. We had to fix two details the paper leaves open."*
1:34 **22 s**		**Slide 6: Enhancement: ADWIN on the Errors** Speaker: Albin John *"Our enhancement watches the model's own mistakes. Every tweet is predicted before it is learned, so each one gives a free right-or-wrong signal. ADWIN raises an alarm when the error rate rises, and the model is rebuilt on the tweets after the change. The OR mode, which combines both detectors, is our main model."*
1:56 **15 s**		**Slide 7: Data and Drift Scenarios** Speaker: Sarthak *"We test on the real stream, the paper's more volatile variant, and six controlled scenarios with known drift points, each with five random seeds. The key one is the label flip: at tweet 40,000 every label inverts while the words stay the same."*
2:11 **10 s**		**Slide 8: Implementation: Tools and Versions** Speaker: Sarthak *"These are the tools and their versions. The Python packages are pinned in the requirements file, and the Spark image is built from our Dockerfile."*
2:21 **5 s**		**Slide 9: Live Demo** Speaker: Albin John *"Now let us show the pipeline running end to end."*
2:26 **95 s**	**LIVE DEMO** terminals + Grafana	**Live demo**, step by step in Section 2. Speaker: Albin John

4:01 **15 s**		**Slide 10: Results: Baseline Reproduction** Speaker: Sarthak *"First, the baseline reproduces the paper: the model that never forgets is worst, fixed forgetting helps, and informed adaptation is best on both streams. One difference: in our runs, forgetting only on detected change beats constant forgetting."*
4:16 **18 s**		**Slide 11: Results: The Label Flip** Speaker: Sarthak *"On the label flip, the word detector waits a full window, about four thousand tweets, while accuracy sits near twenty-five percent. ADWIN fires after thirty-nine tweets, recovery is two and a half times faster, and accuracy rises from seventy-one to almost seventy-four percent."*
4:34 **20 s**		**Slide 12: Results: All Streams and Real Data** Speaker: Aswin Paramesh *"ADWIN also catches a flip limited to one topic, which the baseline misses in two of five runs. On real data, accuracy improves on both streams, and Sentiment140's change point is caught after seven hundred tweets instead of nineteen thousand. The price is more false alarms, mostly from the word detector."*
4:54 **12 s**		**Slide 13: Performance** Speaker: Aswin Paramesh *"The cost is small. In our benchmark the pipeline kept up with two thousand tweets per second, and ADWIN adds only about ten milliseconds per thousand tweets."*
5:06 **12 s**		**Slide 14: Limitations and Future Work** Speaker: Aswin Paramesh *"Limitations: the labels come from emoticons, the controlled drifts are synthetic, and the model runs on the Spark driver. Next, we would try a context-aware classifier and recent, event-driven data."*
5:18 **12 s**		**Slide 15: Conclusion** Speaker: Aswin Paramesh *"To conclude: watch the errors, not only the words. Combining both detectors catches meaning drift about a hundred times sooner and improves accuracy, at almost no cost, in a real streaming pipeline."*
5:30 **3 s**		**Slide 16: References** Speaker: Aswin Paramesh *"These are our references."*

5:33 **4 s**		**Slide 17: Thank You** Speaker: Aswin Paramesh *“Thank you.”*

2. The live demo, step by step (about 1:35)

Prepare before you press record (10 minutes earlier)

1. Start **Docker Desktop** and wait for *Engine running*. In PowerShell, in the project folder:

   ```
   docker compose up -d
   docker ps
   ```

   Six containers must be up: kafka, kafka-ui, spark-master, spark-worker, influxdb, grafana.

2. Open **Terminal A** (PowerShell, project folder) and start the Spark consumer. It keeps this window busy:

   ```
   docker exec -it spark-master /opt/spark/bin/spark-submit --master spark://spark-master:7077 --conf spark.driver.host=spark-master --conf spark.executorEnv.PYTHONPATH=/opt/project /opt/project/pipeline/spark_consumer.py
   ```

   Wait until it prints `consuming 'tweets' from kafka:29092 with models ['baseline', 'enhanced']`.

3. Open **Terminal B** (PowerShell, project folder) for the producer. Do one short dry run so Spark is warm:

   ```
   .\.venv\Scripts\python.exe pipeline\producer.py --stream s_label_flip --limit 5000 --rate 1000
   ```

4. Open the dashboard in the browser, logged in (user `admin` and your team's Grafana password):

   ```
   http://localhost:3000/d/drift-stream/drift-stream?orgId=1&from=now-2m&to=now&refresh=5s
   ```

   This shows the last 2 minutes and refreshes every 5 s. Close other tabs and notifications; set the browser zoom to 100 %. Optional: add `&kiosk` to the URL to hide Grafana's menus.

5. Arrange the windows so you can switch quickly: Terminal A, Terminal B, browser. Use a large terminal font (Ctrl + mouse wheel).

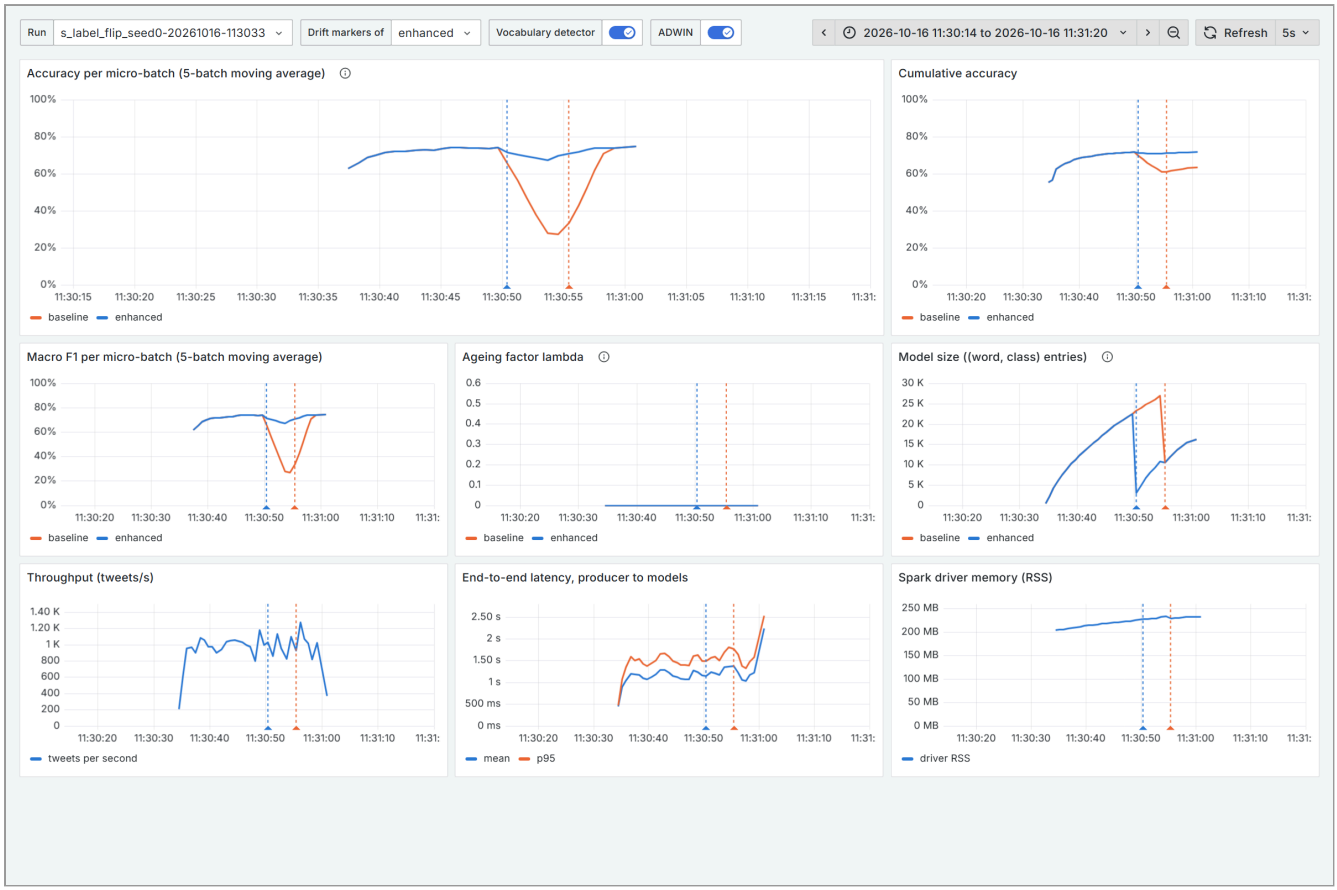
During the recording

Time	Window	Do	Say
+0:00	**Terminal B**	Run `docker ps --format "table {{.Names}}\t{{.Status}}"`. Six containers are listed; kafka and influxdb show *healthy*.	*"All six services run in Docker: Kafka, Kafka UI, the Spark master and worker, InfluxDB and Grafana."*
+0:10	**Terminal A**	Show the consumer that is already running (started before recording). The last line reads `consuming 'tweets' from kafka:29092 with models ['baseline', 'enhanced']`.	*"The Spark consumer is subscribed to the Kafka topic and runs the baseline and our enhanced model side by side."*
+0:20	**Terminal B**	Run `.\.venv\Scripts\python.exe pipeline\producer.py --stream s_label_flip --start 25000 --limit 25000 --rate 1000` and immediately switch to the browser.	*"We replay the twenty-five thousand tweets around the label flip, at one thousand tweets per second. Fifteen seconds in, every label inverts while the words stay the same."*

+0:28	**Grafana**	Press F5 once, then make sure the **Run** menu shows the newest `s_label_flip_seed0-...` run. Point at the accuracy panel, then throughput and latency.	*"Both models learn quickly and reach about seventy-five percent. Throughput is about one thousand tweets per second, with a delay of one to two seconds."*
+0:45	**Grafana**	About 15 s after the producer started: the flip. Point at the **blue ADWIN marker**, the drop in the **model size** panel (rebuild) and the **orange line staying near 25 %**.	*"Here is the flip. The blue marker is ADWIN, firing within fifty tweets: the enhanced model rebuilds and stays near seventy percent, while the orange baseline drops to twenty-five percent and waits about five thousand tweets for its word detector, the orange marker."*
+1:05	**Grafana**	When the run ends (25 s), point at the **Cumulative accuracy** panel (top right).	*"Over these tweets, cumulative accuracy ends at about seventy-two percent for ours against sixty-three for the baseline. The gap is what the error-based detector buys us."*
+1:15	**Terminal B**	Run `.\.venv\Scripts\python.exe experiments\runner_offline.py --stream s_label_flip --model enhanced --fusion or --strategy Rebuild --lam 0 --w 4000` and scroll through the printed metrics. It takes 10–40 s while Spark is busy: cut the wait in the editor, or run it once just before recording and only scroll through its output on camera.	*"The same core code runs offline for the full experiment grid: one command prints accuracy, F1, detection delay, false alarms, recovery time and cost."*

> **Timing tip:** the flip happens about 15 s after the producer starts and the replay takes 25 s. Grafana refreshes every 5 s, so the alarm appears a few seconds later on screen. Keep the rate at **1,000 tweets/s**: on a freshly started stack the consumer sustains about 1,000/s with 1,000-tweet micro-batches, and at 2,000/s it falls behind and the charts lag (tested). Pauses can be cut out later in the editor (Section 3). To show the full 80,000-tweet stream instead, drop `--start` and `--limit` (80 s; speed that part up 2x in the editor).

What the dashboard looks like at the flip (orange = baseline, blue = enhanced)



Top left: accuracy per micro-batch, with the drift markers (dashed lines; blue = ADWIN, orange = vocabulary detector). Middle right: model size; a sudden drop is a rebuild. Bottom: throughput, latency, driver memory.

3. How to record, edit and export the MP4

Recommended: OBS Studio (free) + Clipchamp (built into Windows 11)

1. Install **OBS Studio** (obsproject.com). In *Settings* → *Output*: Recording format **MP4** (or MKV and use *File* → *Remux Recordings* to get MP4). In *Settings* → *Video*: base and output resolution **1920×1080**, 30 fps. In *Settings* → *Audio*: choose your microphone; check the level meter moves when you speak and stays out of the red.
2. Create two scenes: **Slides** (source: *Window Capture* of the PowerPoint slide show, or *Display Capture*) and **Demo** (source: *Display Capture* of the screen with the terminals and browser).
3. Record in **three clips** so a mistake never forces a full retake:
    - **Clip 1:** slides 1–9 (PowerPoint: *Slide Show* → *From Beginning*; each speaker reads their part).
    - **Clip 2:** the live demo (Section 2).
    - **Clip 3:** slides 10–17 (*Slide Show*, start from slide 10).
4. Open **Clipchamp** (Start menu), create a video, import the three clips, put them on the timeline in order, cut long pauses and dead time (for example while Grafana refreshes), and check the total length is under 6:00. *Export* → *1080p*. Clipchamp exports MP4.
5. Name the file `18_Demo_Video.mp4` and put it in `submission\18_LMS_Upload\`.

Alternative without OBS

- **Slides:** PowerPoint → *Record* (Record from beginning); each speaker records the narration on their slides; then *File* → *Export* → *Create a Video* (Full HD) gives an MP4.
- **Demo:** Xbox Game Bar (**Win + Alt + R** starts and stops recording of the active window, with the microphone on via Win + Alt + M), or the Windows Snipping Tool's video mode.
- Join the clips in Clipchamp as above.

Audio tips

- Use a headset or earphone microphone in a quiet room; turn off fans and notifications.
- Rehearse each part twice with a timer; the planned times are in Section 1.
- Speak while the content is on screen; do not read the bullets word for word, the narration already adds what the slide does not say.

4. Set up and run the project from scratch (any Windows machine)

All commands are for **PowerShell**, run in the project folder. Allow about 45 minutes the first time (downloads of Python packages, the dataset and the Docker images).

4.1 Install the prerequisites

- **Docker Desktop** (WSL 2 backend), with at least 8 GB memory available to Docker.
- **Python 3.12** (python.org; tick *Add to PATH*) and **Git**.

4.2 Get the code and the Python environment

```
git clone https://github.com/CASPER0022/Drift-Aware-Sentiment-Stream.git
cd Drift-Aware-Sentiment-Stream
py -3.12 -m venv .venv
Set-ExecutionPolicy -Scope Process Bypass     # only if activation is blocked
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest -q                                     # expect: 46 passed
```

If you use the submitted zip instead of git, unzip it and `cd` into the folder.

4.3 Download Sentiment140 and build the streams

```
New-Item -ItemType Directory -Force data\raw
Invoke-WebRequest https://cs.stanford.edu/people/alecmgo/trainingandtestdata.zip -OutFile
data\raw\trainingandtestdata.zip
python pipeline\build_ds1.py          # DS1: 1.6M tweets sorted by time (~15 s)
python pipeline\build_ds2.py          # DS2: 1,073,065 tweets
python pipeline\build_scenarios.py    # 6 drift scenarios x 5 seeds (~20 s)
```

4.4 Start the streaming stack

```
Copy-Item .env.example .env
docker compose up -d --build          # first time builds the Spark image (a few minutes)
docker ps                             # 6 containers up
```

Service	Address
Grafana dashboard “Drift Stream”	`http://localhost:3000/d/drift-stream` — first login `admin` / `admin`, then Grafana asks you to set a new password
Kafka UI	`http://localhost:8085`
Spark master UI / running consumer	`http://localhost:8090` / `http://localhost:4040`
InfluxDB	`http://localhost:8086`

4.5 Run the pipeline

```
# Terminal A: Spark consumer (baseline + enhanced side by side); stop with Ctrl+C
docker exec -it spark-master /opt/spark/bin/spark-submit --master spark://spark-master:7077 --conf
spark.driver.host=spark-master --conf spark.executorEnv.PYTHONPATH=/opt/project
/opt/project/pipeline/spark_consumer.py
```

```
# Terminal B: replay a stream into Kafka
.\.venv\Scripts\python.exe pipeline\producer.py --stream s_label_flip --rate 1000
.\.venv\Scripts\python.exe pipeline\producer.py --stream ds1 --start 1250000 --limit 150000 --rate
3000
```

4.6 Offline experiments (no Docker needed)

```
.\.venv\Scripts\python.exe experiments\runner_offline.py --stream s_label_flip --model enhanced --
fusion or --strategy Rebuild --lam 0 --w 4000
python experiments\run_config.py --config experiments\configs\baseline.yaml   # paper reproduction
(~20 min)
python experiments\run_grid.py                                                 # full grid, 224 runs
(~25 min)
python analysis\make_report_figures.py                                         # all report figures
and tables
```

4.7 Stop everything

```
Ctrl+C in Terminal A
docker compose stop          # keeps all data; "docker compose start" resumes
```

5. Troubleshooting

Problem	Fix
Docker commands fail: "cannot find the file specified … dockerDesktopLinuxEngine"	Docker Desktop is not running. Start it, wait for *Engine running*, then `docker compose up -d`.
Grafana shows *No data*	Pick the newest run in the **Run** menu and set the time range to *Last 5 minutes*; check Terminal A prints `batch ...` lines.
Consumer prints a Python traceback	`docker restart spark-master spark-worker`, wait 10 s, start the consumer again (Section 2, step 2).
Latency keeps growing in Grafana	The consumer is behind: lower `--rate` to 1000, or restart the consumer with `--batch-size 5000` added at the end.
`producer.py`: file not found under `data\`	Build the streams first (Section 4.3).
Activate.ps1 is blocked	`Set-ExecutionPolicy -Scope Process Bypass`, then activate again.
Video is longer than 6:00	In Clipchamp trim pauses in the demo (Grafana refresh waits) and the gaps between slides; do not cut the flip.

6. Final checklist

File in `submission\18_LMS_Upload\`	What it is
`18_Demo_Video.mp4`	the recorded video: MP4, at most 6:00, slides + live demo + narration (add after recording)

`18_Presentation.pptx`	the slides, in the course template
`18_Final_Report.pdf`	IEEE two-column report, 8 pages, with names and roll numbers
`18_Final_Report_Source.zip`	editable LaTeX source of the report (main.tex, refs.bib, figures, tables)
`18_Source_Code.zip`	complete source code with the README (pipeline, setup, run instructions, tools and versions)
`18_README.md`	the README on its own, for quick reading

- ☐ Video plays from start to end, audio is clear, length ≤ 6:00, file is MP4.
- ☐ Every file name starts with `18_`.
- ☐ The team lead uploads all files to the LMS before the deadline.
- ☐ Everyone has rehearsed the Q&A topics: why naive Bayes, why ADWIN, how delay is measured, why Spark when the model is sequential, how the drifts were made.